

Cloud-Based File Storage System with Nextcloud: Design, Deployment, and Evaluation

Omar Cusma Fait

May 2026

Abstract

This report presents the design, implementation, and evaluation of a self-hosted cloud-based file storage system using Nextcloud, orchestrated with Docker Compose. The project integrates robust monitoring through Prometheus and Grafana, alongside performance testing with Locust. By leveraging containerization, the system demonstrates core functionalities such as user authentication, private storage spaces, and file operations while addressing scalability, security, and cost-efficiency considerations. Insights from the deployment highlight both achievements and opportunities for enhancement in line with modern cloud computing principles.

1 Introduction

In the evolving landscape of cloud computing, self-hosted solutions offer significant advantages in terms of data sovereignty, customization, and cost control. This project focuses on deploying a cloud-based file storage system that enables users to securely upload, download, and manage files within private storage spaces. Nextcloud was selected as the core platform due to its mature open-source ecosystem, native support for the WebDAV protocol for file operations, and built-in user management capabilities.

The implementation emphasizes containerization with Docker Compose to ensure reproducibility and ease of deployment across environments. A comprehensive observability stack, including Prometheus for metrics collection and Grafana for visualization, provides real-time insights into system performance. Load testing with Locust simulates concurrent user activity, allowing evaluation of the system's behavior under stress. This work aligns with key cloud computing principles, exploring how containerized deployments can balance functionality, scalability, and efficiency in a local development setting, while laying the groundwork for production cloud environments. The project encourages curiosity about the interplay between open-source tools and DevOps practices in building resilient cloud applications.

2 Project Structure

The project is organized into a modular directory structure that separates configuration, data, documentation, and testing components. At its heart lies the `docker-compose.yml` file, which defines five primary services: Nextcloud as the main application server, a dedicated Nextcloud Exporter for metrics exposure, Prometheus for time-series data storage, Grafana for dashboarding, and Locust for load generation. Persistent data is managed through Docker volumes for Nextcloud files, Prometheus storage, and Grafana configurations, ensuring durability across container restarts.

Supporting elements include environment variable files for credential management, a custom Locust tasks script (`tasks.py`) that implements WebDAV-based upload, download, and delete operations with randomized file sizes, and provisioning configurations for Grafana datasources.

Documentation files such as `execution_log.md` and `README.md` provide detailed guidance, while backup versions of Locust scripts illustrate iterative development. This structure promotes maintainability and facilitates experimentation with different file sizes or task distributions, reflecting best practices in reproducible research and infrastructure-as-code.

3 Launching the System

Launching the system begins with prerequisite setup on a Windows 11 environment utilizing WSL2 and Docker Desktop. Administrators should open a command prompt in elevated mode, activate any relevant virtual environment, and ensure Docker is running. The primary startup command is

```
docker compose -env-file .\env\.env up -d -build,
```

which orchestrates the entire stack, pulling pinned images (using SHA256 digests for reproducibility) and building necessary components.

Verification involves checking container status with `docker compose ps` and accessing individual services: Nextcloud at `http://localhost:8080` for user management and file operations, Prometheus targets at `http://localhost:9090/targets` to confirm the exporter is UP, Grafana at `http://localhost:3000`, and Locust at `http://localhost:8089`. Users are created via the Nextcloud admin interface, granting each a private WebDAV endpoint at `/remote.php/dav/files/username/`. Credentials are securely managed through the `.env` file, and internal Docker networking allows seamless communication between services using service names as hostnames. This streamlined process highlights the power of Docker Compose in simplifying complex multi-container deployments.

4 Monitoring

Effective monitoring is central to understanding system behavior in dynamic cloud environments. The Nextcloud Exporter scrapes internal metrics from the application and exposes them in Prometheus format at `http://localhost:9205/metrics`. Prometheus, configured with a 30-second scrape interval via `prometheus.yml`, collects these metrics into its time-series database, while Grafana provisions the Prometheus datasource automatically for intuitive visualization.

Key dashboards track metrics such as `nextcloud_up` for health status, active user counts, total files, scrape durations, and request rates derived from `nextcloud_php_request_duration_seconds`. This observability pipeline enables proactive identification of bottlenecks, such as increased latency during high-load scenarios, and aligns with cutting-edge practices in infrastructure monitoring. Exploring Grafana's query language (PromQL) offers fascinating opportunities to derive custom insights, such as correlating file operations with resource utilization.

5 Results and Discussion

5.1 Load Testing

Load testing was conducted using Locust to simulate realistic WebDAV interactions. The `tasks.py` script defines a `WebDavUser` class that authenticates via Basic Auth and performs randomized upload operations with files generated by `os.urandom()` for incompressible data, followed by cleanup. Tests with 30 concurrent users and a ramp-up rate of 2 users per second yielded approximately 15 requests per second, with median response times around 15 ms across PUT, GET, and DELETE operations for small files (50 KB).

The results of the test period (approximately 72 seconds) showed consistent performance for small files, although initial runs encountered 401 Unauthorized errors resolved through proper credential configuration and timeouts. Larger files increase I/O demands, suggesting potential

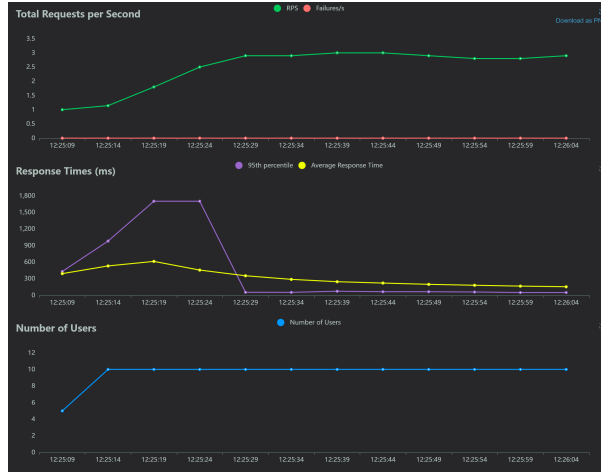


Figure 1: Performance results from Locust load test with 10 users and 500 kB files.

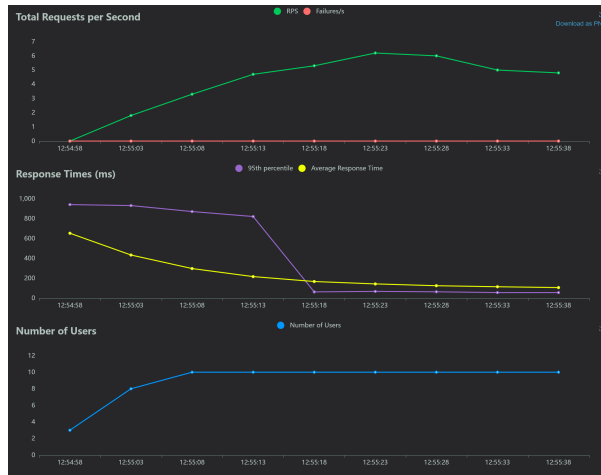


Figure 2: Performance results from Locust load test with 10 users and 5 MB files.

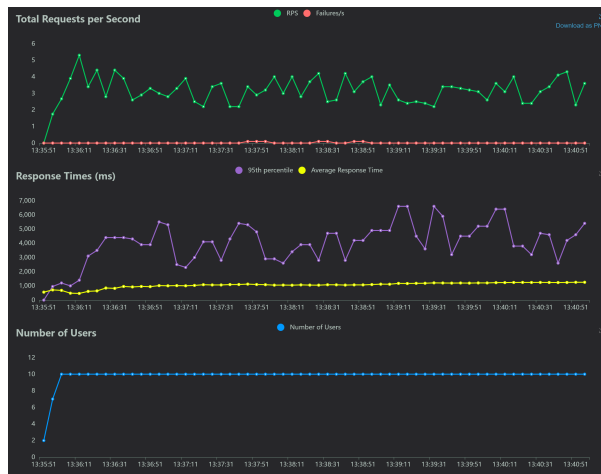


Figure 3: Performance results from Locust load test with 10 users and 50 MB files.

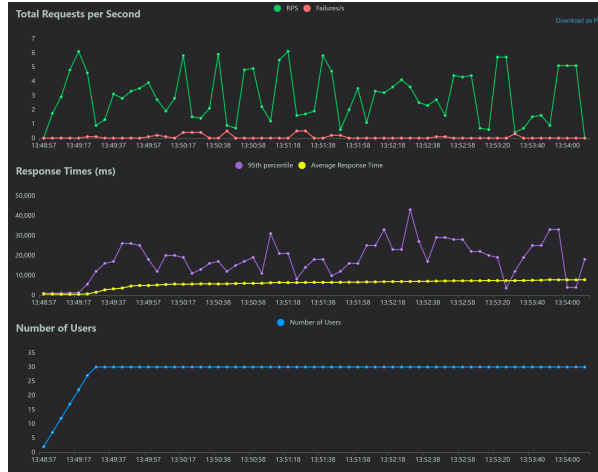


Figure 4: Performance results from Locust load test with 30 users and 50 MB files.

bottlenecks in disk throughput or PHP-FPM workers. These findings underscore the importance of task weighting and wait times in load scripts for accurate simulation of user behavior. Future tests could incorporate distributed Locust workers for higher concurrency.

5.2 Cost-Efficiency

The deployment prioritizes cost-efficiency by relying exclusively on open-source components and lightweight containers, incurring zero licensing fees. Docker volumes optimize storage usage, while efficient monitoring reduces unnecessary overhead. In a local context, this setup minimizes resource consumption on commodity hardware. For production, theoretical migration to cloud providers like AWS (using EC2, S3 for object storage, and managed services such as RDS PostgreSQL) would leverage pay-as-you-go models and auto-scaling to further optimize expenses, particularly through spot instances during low-demand periods. This approach demonstrates thoughtful consideration of total cost of ownership in cloud architectures.

6 Improvements

6.1 Security

Although the current setup employs Basic Auth over WebDAV and Nextcloud’s built-in access controls, several enhancements could strengthen the security posture. Implementing TLS termination via a reverse proxy such as Traefik or Nginx would secure data in transit. Additional measures include enabling multi-factor authentication, configuring strict password policies, and utilizing Nextcloud’s encryption capabilities for data at rest. Network isolation within Docker, combined with regular security updates and vulnerability scanning, aligns with industry standards. Exploring zero-trust architectures or integrating with external identity providers presents intriguing directions for enterprise-grade deployments.

6.2 Scalability

The system performs adequately for moderate loads with small-to-medium files but would benefit from architectural refinements for larger scales. Horizontal scaling could involve multiple Nextcloud application instances behind a load balancer, paired with shared object storage and database clustering. Nextcloud’s Global Scale architecture offers advanced sharding and federation capabilities for massive user bases. Vertical scaling through increased container resources and PHP optimization provides immediate gains. Future work might investigate Kubernetes

orchestration for automated scaling and resilience, opening exciting avenues for high-availability cloud storage solutions.

7 References

- Nextcloud Official Documentation.
- Prometheus and Grafana Deployment Guides.
- Locust Performance Testing Documentation.
- Project repository files including `execution_log.md`, `docker-compose.yml`, and test results.